Para todos los ejercicios:

Requisitos técnicos y de calidad

- **Java 17+** y **Maven**.
- **Modulariza** la solución en **clases separadas** y **paquetes**
- Buenas prácticas:
    - Encapsulación (atributos privados + getters/setters).
    - Path/Files y creación de directorios si no existen.
    - try-with-resources en operaciones de E/S.
    - Validación básica de entradas (no vacíos, números ≥ 0, fecha ISO).
    - Sin usar break/continue para salir de bucles.
    - Documentación interna y javadoc

Ejercicio 1 Gestión biblioteca DOM

Una biblioteca digital desea mantener un registro actualizado de los libros almacenados en formato XML. El archivo actual, biblioteca.xml, contiene información básica de cada libro: su identificador (id), título, autor y año de publicación.
Sin embargo, la estructura necesita algunas modificaciones: algunos libros no tienen el atributo de edición (edicion), y el personal de la biblioteca quiere añadir nuevos títulos.

Tu tarea consiste en **leer, analizar, modificar y guardar** el archivo XML utilizando el **modelo DOM (Document Object Model)**, que representa el documento como un árbol de nodos en memoria.

Tareas a realizar:

1. **Cargar** el fichero XML en memoria utilizando un DocumentBuilder del paquete javax.xml.parsers.
    - Normaliza el documento con doc.normalize() para evitar nodos de texto duplicados.
2. **Recorrer** la estructura del árbol DOM:
    - Lista por consola todos los libros mostrando su id, título, autor y año en el formato:

```
[L01] El Quijote (1605) – Miguel de Cervantes
[L02] La Regenta (1884) – Leopoldo Alas «Clarín»
```

3. **Modificar datos existentes**:
    - Cambia el año (<anio>) del libro con id="L02" a 1885.
    - Si algún libro **no tiene el atributo edicion**, añádelo con el valor "1".
4. **Añadir un nuevo libro al final del documento**:
    - Crea un nuevo elemento <libro> con atributos y subelementos:

```
<libro id="L03" edicion="1">
    <titulo>Fortunata y Jacinta</titulo>
    <autor>Benito Pérez Galdós</autor>
    <anio>1887</anio>
</libro>
```

    - Inserta este nodo como hijo del elemento raíz <biblioteca>.
5. **Guardar el resultado en un nuevo archivo XML** llamado biblioteca_modificada.xml.
    - Usa un Transformer para escribir el árbol DOM al disco con formato legible (sangrado).
6. **Mostrar un mensaje de confirmación** en consola indicando la ruta absoluta del archivo generado.

Fichero biblioteca.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<biblioteca>
    <libro id="L01" edicion="1">
        <titulo>El Quijote</titulo>
        <autor>Miguel de Cervantes</autor>
        <anio>1605</anio>
    </libro>
    <libro id="L02">
        <titulo>La Regenta</titulo>
        <autor>Leopoldo Alas «Clarín»</autor>
        <anio>1884</anio>
    </libro>
</biblioteca>
```

Ampliaciones opcionales

- Contar el número total de libros del documento.
- Calcular el rango de años (mínimo y máximo).
- Añadir un atributo adicional pais="España" a todos los libros del XML.
- Usar getElementsByTagName() y getAttribute() correctamente para practicar el acceso a nodos.

Ejercicio 2 Gestión alumnos operaciones CRUD

El instituto desea mantener un registro de los alumnos matriculados en un curso de Formación Profesional en formato XML.
El fichero actual, alumnos.xml, almacena un listado con el identificador, nombre y edad de cada estudiante.
El personal administrativo necesita una pequeña herramienta que permita **añadir, actualizar y eliminar alumnos** del fichero, para mantenerlo siempre actualizado.

Tu objetivo es desarrollar un programa Java que gestione dicho fichero usando el **modelo DOM (Document Object Model)**, permitiendo realizar las operaciones básicas de un CRUD (*Create, Read, Update, Delete*) sobre el árbol XML.

Fichero alumnos.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<alumnos>
    <alumno id="A01">
        <nombre>Lucía</nombre>
        <edad>21</edad>
    </alumno>
    <alumno id="A02">
        <nombre>Hugo</nombre>
        <edad>20</edad>
    </alumno>
</alumnos>
```

Tareas a realizar

1. **Cargar el fichero XML en memoria** usando un DocumentBuilder y normalizar el documento.
    - Comprueba que el nodo raíz es <alumnos> y que los elementos <alumno> contienen los subelementos <nombre> y <edad>.
2. **CREATE – Añadir un nuevo alumno**
    - Crea un nuevo elemento <alumno> con los siguientes datos:

```
<alumno id="A03">
```

```
    <nombre>Sara</nombre>
    <edad>22</edad>
</alumno>
```

- Inserta este nuevo nodo al final del elemento raíz <alumnos>.

3. **READ – Mostrar información actual**
    - Recorre el árbol DOM e imprime por consola todos los alumnos en formato:

```
[A01] Lucía (21 años)
[A02] Hugo (20 años)
[A03] Sara (22 años)
```

4. **UPDATE – Modificar un dato existente**
    - Busca el alumno con id="A02" y cambia su edad (<edad>) a **21**.
5. **DELETE – Eliminar un alumno**
    - Localiza el nodo <alumno id="A01"> y elimínalo completamente del árbol.
6. **Guardar el resultado final** en un nuevo archivo XML llamado alumnos_out.xml.
    - Utiliza un Transformer para escribir el documento con formato indentado y legible.
    - Muestra un mensaje en consola indicando el número de alumnos restantes y la ruta donde se ha guardado el fichero.

Ampliaciones opcionales

- Permitir que los datos del nuevo alumno se pidan al usuario por consola mediante Scanner.
- Añadir una etiqueta <curso> dentro de cada <alumno> con el nombre del ciclo formativo.
- Calcular y mostrar la edad media del grupo.
- Ordenar los alumnos por edad antes de guardar el fichero resultante.

Ejercicio 3 Contar alumnos mayores de edad

A diferencia del modelo DOM, el modelo **SAX (Simple API for XML)** no carga todo el documento XML en memoria.
En su lugar, analiza el fichero **secuencialmente**, lanzando **eventos** cada vez que se abre o cierra una etiqueta, o se encuentra texto en su interior.

Este enfoque es ideal cuando los documentos XML son grandes y sólo necesitamos **leer datos sin modificarlos**.

En este ejercicio, utilizarás SAX para **contar cuántos alumnos tienen 21 años o más** a partir del fichero alumnos.xml, que contiene información básica sobre el alumnado de un ciclo formativo.

Archivo alumnos.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<alumnos>
    <alumno id="A01">
        <nombre>Lucía</nombre>
        <edad>21</edad>
    </alumno>
    <alumno id="A02">
        <nombre>Hugo</nombre>
        <edad>20</edad>
    </alumno>
    <alumno id="A03">
        <nombre>Sara</nombre>
        <edad>22</edad>
    </alumno>
</alumnos>
```

Tareas a realizar

1. **Preparar el parser SAX**
    - Crea una instancia de SAXParserFactory y genera un SAXParser.
    - Define una clase manejadora (por ejemplo, ManejadorAlumnos) que herede de DefaultHandler.
2. **Implementar los métodos del manejador**
    - startElement(...) → detecta la apertura de etiquetas.
    - characters(...) → captura el texto dentro de las etiquetas <nombre> y <edad>.
    - endElement(...) → al cerrar una etiqueta, determina si debe procesar el contenido leído.
3. **Procesar los datos**
    - Cada vez que se cierra una etiqueta <alumno>, analiza la edad del alumno.
    - Si la edad es **mayor o igual que 21**, guarda su nombre en una lista o contador.
4. **Mostrar resultados por consola**

- Muestra el número total de alumnos mayores o iguales a 21 años.
- Lista los nombres de dichos alumnos.
- Ejemplo de salida:

```
Mayores o iguales a 21: 2
Nombres: [Lucía, Sara]
```

5. **Control de errores**
   - Añade bloques try-catch para capturar posibles errores de lectura o formato.
   - Asegúrate de eliminar espacios en blanco al procesar texto con trim().

Ampliaciones opcionales

- Mostrar también la **media de edad** del grupo.
- Contar cuántos alumnos hay en total y calcular el **porcentaje de mayores de edad**.
- Añadir una opción que lea el **nombre del archivo XML** desde los argumentos del programa.
- Incluir el **id** del alumno en la lista de salida, por ejemplo: [A01 - Lucía, A03 - Sara].

Ejercicio 4 Suma y media precios en un catálogo

Dispones de un fichero catalogo.xml con productos de una tienda online. Cada producto tiene un nombre y un precio.
Tu objetivo es **leer el XML de forma secuencial con SAX** para calcular:

- el **número de productos**,
- la **suma total** de los precios,
- y la **media**.

Este ejercicio refuerza el manejo de **eventos SAX** y el tratamiento de datos numéricos extraídos como texto.

Fichero catalogo.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<catalogo>
    <producto>
        <nombre>Teclado</nombre>
        <precio>19.99</precio>
```

```
    </producto>
    <producto>
        <nombre>Ratón</nombre>
        <precio>12.50</precio>
    </producto>
    <producto>
        <nombre>Pantalla</nombre>
        <precio>149.00</precio>
    </producto>
</catalogo>
```

Tareas a realizar

1. **Configurar el parser SAX**
    - Crea SAXParserFactory y SAXParser.
    - Implementa una clase ManejadorCatalogo que extienda DefaultHandler.
2. **Capturar contenido de <precio>**
    - En startElement(...) y endElement(...) controla cuándo entras/sales de la etiqueta <precio>.
    - En characters(...) acumula el texto (puede llegar por fragmentos).
    - Al cerrar <precio>, convierte el texto a double y **súmalo** a un acumulador.
3. **Contar productos**
    - Incrementa un contador cada vez que comience un <producto> (en startElement(...)).
4. **Calcular estadísticas**
    - Tras el parseo, calcula la **media**
5. **Mostrar resultados**
    - Muestra por consola:

    ```
    Productos: N
    Suma precios: S
    Media: M
    ```

    - Formatea a dos decimales.
6. **Robustez**
    - Usa trim() al leer textos.

    - Maneja posibles NumberFormatException (precios mal formateados).
    - Considera la **configuración regional** (el XML usa punto decimal).

Ampliaciones opcionales

- Mostrar también el **precio mínimo** y **máximo** encontrados.
- Listar los **nombres** de los productos cuyo precio esté **por encima de la media**.
- Aceptar el **nombre del fichero** como argumento de línea de comandos.
- Soportar un **atributo moneda** opcional en <precio moneda="EUR"> e incluirlo en el resumen.

Ejercicio 5 Curso con alumnos

El departamento de informática del instituto mantiene un listado de cursos y alumnos matriculados.

Hasta ahora, la información se gestionaba manualmente en un archivo XML, pero se desea automatizar el proceso de generación y lectura de los datos desde aplicaciones Java.

Para ello, vas a utilizar Jakarta JAXB, una tecnología que permite convertir automáticamente objetos Java ↔ XML sin recorrer las etiquetas manualmente.

Este modelo se conoce como data binding: el programa trabaja con objetos, y JAXB se encarga de crear o interpretar el XML según las anotaciones (@XmlRootElement, @XmlElement, etc.) definidas en las clases.

Estructura de clases:

Clase Alumno

- Atributos:
    - id (atributo XML).
    - nombre y edad (elementos XML).
- Anotaciones:
    - @XmlAttribute → para el identificador.
    - @XmlElement → para los demás campos.

Clase Curso

- Atributos:
    - anyo (atributo XML).
    - codigo, nombre (elementos XML).
    - List<Alumno> → lista de alumnos.
- Anotaciones:
    - @XmlRootElement(name="curso")
    - @XmlAccessorType(XmlAccessType.FIELD)
    - @XmlElementWrapper(name="alumnos")
    - @XmlElement(name="alumno")

Tareas a realizar

1. **Crear las clases del modelo (Alumno y Curso)**
    - Implementa las clases con sus atributos, constructores, getters y setters.
    - Añade las anotaciones JAXB necesarias para definir su correspondencia con el XML.
2. **Configurar el proyecto Maven**
    - Crea un fichero pom.xml con la dependencia:

```
<dependency>
  <groupId>org.glassfish.jaxb</groupId>
  <artifactId>jaxb-runtime</artifactId>
  <version>4.0.5</version>
</dependency>
```

    - Define source/target para Java 17 o 11.
3. **Generar un objeto Curso en Java**
    - Crea un objeto con los datos:
        - anyo = 2025
        - codigo = DAM-AD
        - nombre = Acceso a Datos

- Añade varios alumnos al curso:

```
new Alumno("A01", "Lucía", 21)
new Alumno("A02", "Hugo", 20)
```

4. **Marshalling (Java → XML)**
   - Usa un Marshaller para generar el fichero curso.xml.
   - Configura el formato legible con:

```
marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
```

   - Guarda el resultado en el disco y muestra también por consola el XML generado.
5. **Unmarshalling (XML → Java)**
   - Crea un Unmarshaller para leer el fichero curso.xml.
   - Muestra por consola el código del curso y el número de alumnos recuperados.
6. **Comprobación de salida esperada**

Ampliaciones opcionales

- Permitir añadir más alumnos leyendo sus datos desde consola.
- Añadir un nuevo campo tutor al curso con anotación @XmlElement.
- Implementar un método para calcular la **edad media del grupo** y mostrarla al final.
- Crear una clase auxiliar Centro que contenga varios cursos, para practicar anidación de listas (@XmlElementWrapper dentro de otra clase).

Ejercicio 6 Biblioteca con enumeraciones y composición de objetos

Continuando con el uso de **Jakarta JAXB** para el intercambio de datos entre objetos Java y documentos XML, se plantea ahora un ejemplo más avanzado: un sistema para gestionar la información de una **biblioteca digital**, en la que cada **obra** tiene distintos tipos (libro, cómic, revista) y un **comentario textual**.

El objetivo es demostrar cómo JAXB permite **representar estructuras más complejas** en XML, incluyendo:

- Enumeraciones (enum) con valores personalizados.

- Clases anidadas (composición).
- Control del orden de las etiquetas (@XmlType(propOrder)).
- Representación de texto simple dentro de una etiqueta (@XmlValue).

Estructura de clases

TipoObra.java

Representa el tipo de cada obra mediante una enumeración.

```
@XmlEnum
public enum TipoObra {
    @XmlEnumValue("LIBRO") LIBRO,
    @XmlEnumValue("REVISTA") REVISTA,
    @XmlEnumValue("COMIC") COMIC
}
```

Comentario.java

Representa un texto libre dentro de una etiqueta <comentario>.

```
public class Comentario {
    @XmlValue
    private String texto;
}
```

Obra.java

Define los datos de cada obra de la biblioteca.

- Atributos: id, titulo, autor, anioPublicacion, tipo, comentario.
- Usa @XmlType(propOrder={...}) para mantener el orden de los elementos.

```
@XmlType(propOrder={"titulo", "autor", "anio", "tipo", "comentario"})
public class Obra {
    @XmlAttribute private String id;
    @XmlElement private String titulo;
    @XmlElement private String autor;
    @XmlElement(name="anioPublicacion") private int anio;
    @XmlElement private TipoObra tipo;
    @XmlElement private Comentario comentario;
}
```

Biblioteca.java

Clase raíz que contiene una lista de obras.

```
@XmlRootElement(name="biblioteca")
@XmlAccessorType(XmlAccessType.FIELD)
public class Biblioteca {
    @XmlElementWrapper(name="obras")
    @XmlElement(name="obra")
    private List<Obra> obras;
}
```

areas a realizar

1. **Crear las clases del modelo**
    - Implementa TipoObra, Comentario, Obra y Biblioteca con las anotaciones mostradas.
    - Asegúrate de incluir constructores sin argumentos y getters/setters cuando sea necesario.
2. **Crear y rellenar una biblioteca**
    - En la clase MainBiblioteca, crea una instancia de Biblioteca y añade varias obras:

```
b.addObra(new Obra("O01", "Cien años de soledad", "G.G. Márquez", 1967,
                   TipoObra.LIBRO, new Comentario("Imprescindible en
narrativa hispanoamericana")));
b.addObra(new Obra("O02", "Astérix en Hispania", "Uderzo & Goscinny", 1969,
                   TipoObra.COMIC, new Comentario("Clásico del cómic
europeo")));
```

3. **Realizar el proceso de marshalling (Java → XML)**
    - Crea el Marshaller a partir del JAXBContext.
    - Activa el formato legible con marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true).
    - Muestra el resultado por consola y guárdalo en un archivo biblioteca.xml.
4. **Realizar el proceso de unmarshalling (XML → Java)**
    - Crea un Unmarshaller para leer biblioteca.xml y reconstruir el objeto Biblioteca.
    - Muestra por consola cuántas obras se leyeron correctamente.

5. **Verifica que la salida generada coincide con la estructura esperada.**

Ampliaciones opcionales

- Añadir un nuevo tipo de obra al enum (ENSAYO o POESIA).
- Incorporar un campo prestado (boolean) a la clase Obra para indicar si está disponible.
- Crear un método que cuente cuántas obras hay de cada tipo (LIBRO, COMIC, REVISTA).
- Generar una versión JSON del mismo objeto utilizando la librería Gson para comparar estructuras.
- Añadir anotación @XmlElementWrapper(name="comentarios") si cada obra tuviera varios comentarios.